

AI Planning for Robot Systems

Coursework 1 Report



Group Members

Student ID	Name
235091	Abdelrahman
230269	Kareem Zakaria
232575	Asser Ahmed
229022	Marwan Galil

Semester One, 2024–2025

Contents

1	Introduction	1
2	System Pipeline Overview	2
3	Design Approach	4
3.1	Algorithms	4
3.2	Navigation and Obstacle Avoidance	4
4	Implementation Details	6
5	Challenges and Solutions	7
6	Testing and Results	8
6.1	Situations	8
6.2	Test Results	8
7	Conclusion	10

List of Figures

2.1	System pipeline architecture showing the data flow between manual setup, exploration, planning, and execution modules.	3
6.1	Combined performance comparison: A* vs Dijkstra showing computation time and path length across 20 test cases.	8
6.2	Detailed performance comparison table showing execution time and path length for A* and Dijkstra algorithms across 20 test cases.	9

Chapter 1

Introduction

The Warehouse Order Fulfillment Robot project implements a path-planning system whereby a robot learns within a warehouse environment to acknowledge priorities for obtaining items. The intention of this project is to create a robot using AI learning from the environment in obstacle avoidance and goal completion fields, constantly adjusting its own path to obtain the highest prioritized goals within the shortest time duration. The environment of this project is an artificial warehouse grid system generated via Python and Pygame. Each square of the warehouse is an occupancy value created by the process of OGM where every space is either 0 (clear) or 1 (obstacle/occupied). Two pathfinding algorithms were explored - Dijkstra's Algorithm and A* (A-star) - to measure their performance moving from the starting point to the desired goals and comparing them. The comparison between both aims to assess effective performance due to static or dynamic situational settings. In this case, there are multiple items in the warehouse environment upon startup with each item tagged with a priority marker 1-5. The robot will always aim for the highest priority first and then orient itself to the second-highest. The robot will also go from Level 1 to Level 3 before accessing another part of the warehouse for low/medium priority other items. This is significant because it indicates that not all shelves/packages/objects carry the same value and only some need to be addressed and considered worthwhile, while others are not worth the time unless on the way. This mimics a realistic, complicated scenario as there are dynamic obstacles (moving humans) that constantly shift location and force redirection and path adjustment in real-time. It is also a generalized solution as many robots could operate in this environment, but may require robotic intelligence to keep refocusing within one area without humans becoming confused.

Chapter 2

System Pipeline Overview

The overall system functionality can be categorized as follows:

1. **Manual Input:** The human user gives an OGM the warehouse image and it segments the environment into a grid with user manually providing where static obstacles are to formulate a binary map which is saved as `manual_ogm.npy`.
2. **Exploration/Map Creation:** The Robot moves about the environment exploring as if it has range sensors (with an in-game mapped log-odds map) with areas subsequently classified as free or occupied through learned probabilistic representation.
3. **Inference-Path Estimation:** The Robot estimates the best path to any target (given) via A* or Dijkstra's Algorithm.
4. **Inference-Replanning:** If the path to the target is covered by a moving worker or obstacle that blocks the pre-determined path, automatic inference detection takes place and re-planning occurs.
5. **Precedence of Significance:** The Robot operates on the next destination from most significant to least significant since all determined targets are recognized, but only one will be moved to at a time, starting with the highest.
6. **Visual Output Map Creation:** The Robot employs Pygame to visualize the arrangement of obstacles, path drawn, and subsequent movements of the Robot and Worker.

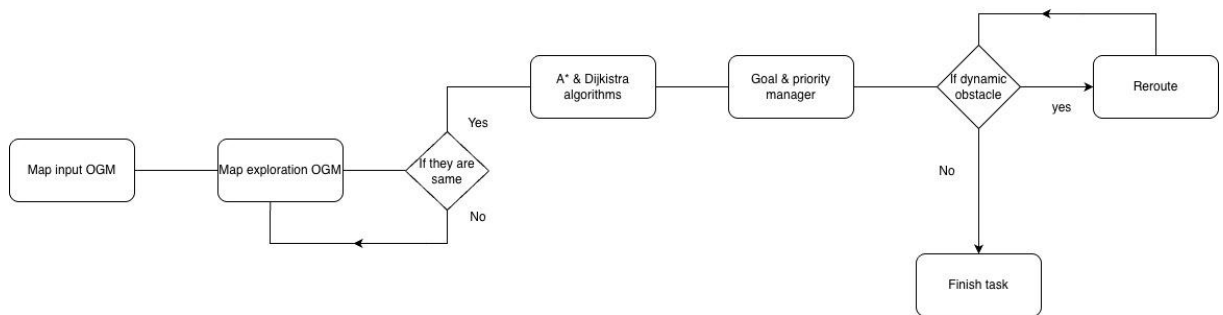


Figure 2.1: System pipeline architecture showing the data flow between manual setup, exploration, planning, and execution modules.

Chapter 3

Design Approach

3.1 Algorithms

The developed system evaluates two classical pathfinding algorithms for performance assessment:

Dijkstra's Algorithm

Dijkstra's Algorithm determines the shortest path from nodes in a weighted graph. It expands nodes based on cost and relies on completed exploration to find an optimal route to the endpoint. As it explores all reachable nodes, it is not ideal for large environments as it takes longer to find solutions.

A* Algorithm

A* Algorithm combines Dijkstra's with a heuristic that builds upon the current node to an endpoint, allowing exploration based on anticipated distance. Heuristic consideration reduces the time taken to render a solution while allowing for near-optimal results. A* is ideal for real-time implementation and dynamic goals.

Comparison

- A* was more efficient in open areas as it expands based on anticipatory heuristic considerations.
- Dijkstra's is more robust and predictable in a diverse environment with many obstructions.
- A* diverted and expanded less than Dijkstra's in anticipation of error correction; however, both required re-routing when workers were added dynamically mid-mission. A* accomplished a given goal faster than Dijkstra's due to less expansion/rendering time for repetitive paths to the goal.

3.2 Navigation and Obstacle Avoidance

Navigation and obstacle avoidance use the OGM map for localization. This means that, for thresholds, 1 is an obstacle and 0 is free, walkable space. Therefore, static obstacles are

the permanent walls and shelves of the space and dynamic obstacles are moving workers who frequent the space. In fact, when a dynamic worker gets close enough within the robot's threshold for obstructing an agent, it will stop and re-plan. Re-planning occurs using map 3, which uses obstacle avoidance and should not occur with collision. Since the map is updated over time, this means that the robot can continuously accomplish its task, no matter what is going on around it. The optimization criteria are:

- Minimized overall distance
- Minimized re-planning occurrence
- Collisions with dynamic agents = 0
- Time to goal relative to computation time

Chapter 4

Implementation Details

The entire project is executed in Python with the following major modules and libraries:

- Pygame - for GUI construction, visualization, and event-based simulation.
- NumPy - for rendering the OGM as a matrix and performing path calculations.
- PIL (Pillow) - for accessing map images and rendering them.
- heapq - for creating priority queues within the A* and Dijkstra implementation.

Modules and Key Functionalities

1. **OGM Editor:** Draws manual obstacles to be saved as grid data.
2. **Exploration Module:** Simulates sensor ranges and incrementally learns the unknown spaces with log-odds probability mapping.
3. **Planner:** Implementation of A* and Dijkstra for path finding.
4. **Dynamic Worker Class:** Simulates humans moving, from randomly generated start positions, to randomly chosen endpoints, requiring robots to adapt their pathing.
5. **Robot Class:** Deals with robot movements, re-routing, and accomplishing goal priorities.
6. **Visualization:** Updating visualizations to show robot progress, worker/item/goal gains/losses in real-time.

All modular functionality was achieved in a way that creates a closed feedback loop between sensing, processing, and movement.

Chapter 5

Challenges and Solutions

- **Dynamic Obstacles:** Manual workers in the area would occasionally step into the robot's path. This was resolved through a blocking radius check and dynamic rerouting through A* and Dijkstra.
- **Mapping Discrepancies:** The map created was not the same as OGM started with because OGM boundaries weren't 100% accurate. The fix was to change grid resolution and scaling.
- **Resource Inconsistency:** Variable CPU resource overhead in frequent repathfinding with A* and Dijkstra. The fix is that path replanning only occurs when the robot is in proximity to a region that's been blocked.
- **Confusion:** Frequently, many high-priority regions would be created at once which would cause planning interference. This was resolved by changing the operation so that one important object was dealt with at a time in each priority.

Chapter 6

Testing and Results

6.1 Situations

The system was evaluated in several situations:

- **Stationary environment:** No movement of workers; only obstacle avoidance;
- **Mobile environment:** Workers moving randomly; must recalculate routes;
- **Priority queuing:** Multiple items in a line but with varying degrees of priority.

6.2 Test Results

All relevant data were collated for A* and Dijkstra under the same mapped conditions across 20 instances. Figure 6.1 shows the combined performance comparison graph with trends in computation time and path length for A* and Dijkstra across all instances.

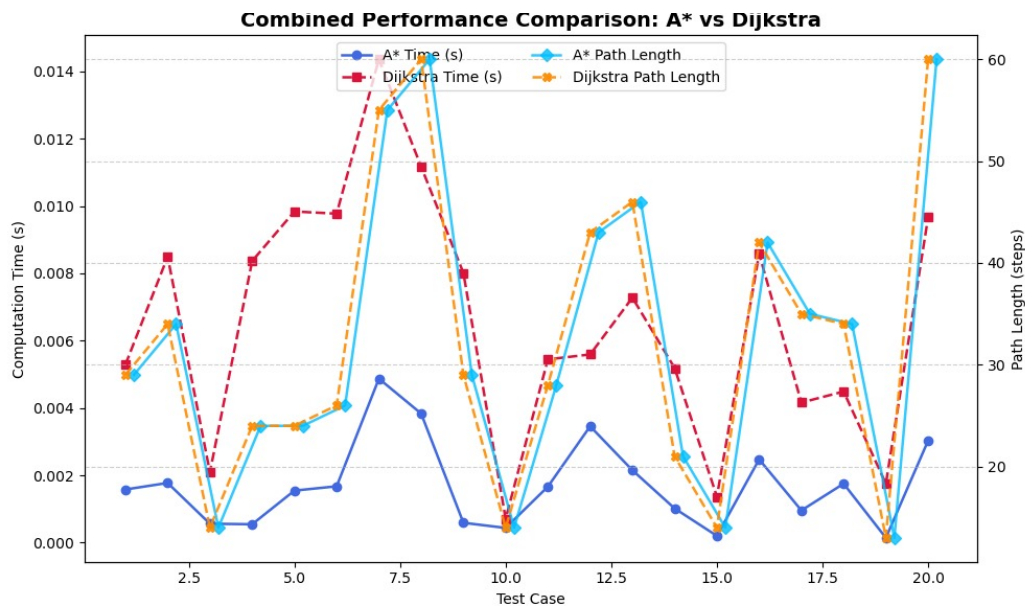


Figure 6.1: Combined performance comparison: A* vs Dijkstra showing computation time and path length across 20 test cases.

The detailed performance comparison table is shown in Figure 6.2.

=== PERFORMANCE COMPARISON TABLE ===					
	Test	A*_Time (s)	A*_Path Length	Dijkstra_Time (s)	Dijkstra_Path Length
0	1	0.001543	29	0.006004	29
1	2	0.001796	34	0.004810	34
2	3	0.000275	14	0.001617	14
3	4	0.000633	24	0.005052	24
4	5	0.000964	24	0.006607	24
5	6	0.000978	26	0.006341	26
6	7	0.004275	55	0.011909	55
7	8	0.005614	60	0.013750	60
8	9	0.001223	29	0.032571	29
9	10	0.000493	14	0.000841	14
10	11	0.002310	28	0.006361	28
11	12	0.005621	43	0.011940	43
12	13	0.006882	46	0.018178	46
13	14	0.001657	21	0.008279	21
14	15	0.000624	14	0.002693	14
15	16	0.006447	42	0.017760	42
16	17	0.001772	35	0.006860	35
17	18	0.001959	34	0.007634	34
18	19	0.000529	13	0.002467	13
19	20	0.005526	60	0.011077	60

Figure 6.2

Figure 6.2: Detailed performance comparison table showing execution time and path length for A* and Dijkstra algorithms across 20 test cases.

A* took less time to finish and had less re-routing which appeared more advantageous in time-sensitive situations, but Dijkstra did result in slightly more smooth paths and more uniformity in densely populated obstacle regions.

Chapter 7

Conclusion

The Warehouse Order Fulfillment Robot possessed intelligent navigation of an unfamiliar environment with perception and dynamic rerouting. Comparison A* and Dijkstra's, in the project's case, relative to other projects indicate that A* is heuristic driven and can produce results quicker meaning time is lessened without sacrificing the optimality of the path. The deliverables of the project were: Real OGM integrated with real time path planning, Dynamic obstacle encouraging path displacement, Task scheduling based on nearest/crossing obstacles as a result of preference given dynamic shopping cart obstacle relative to other obstacles more often. The limitations of the project are that it does not utilize real feedback from sensor information and may get overwhelmed in larger maps with many obstacles. The future work is in developing multi-robot systems, mapping systems (SLAM), reinforcement learning for planners to act without human operator input.